Data transformation

Data Science Flipped Classroom - Module 3

Adriana Clavijo Daza

2026-04-23

Base R vs packages

What is base R?

Base R is the set of functions and tools that come installed with R itself.

It includes:

- Core data types: vectors, lists, data frames
- Basic operations: arithmetic, logic, subsetting
- Built-in functions: `mean()`, `sum()`, `length()`, `summary()`
- Built-in datasets: `iris`, `mtcars`, `airquality`

What is a package?

A package is a collection of functions, data, and documentation that extends R's capabilities.

- Developed by the R community
- Installed from CRAN, GitHub, or other sources
- Must be loaded before use

Installing and loading packages

Install a package **once** with `install.packages()`:

```
install.packages("dplyr")
```

Load it **in every session** with `library()`:

```
library(dplyr)
```

The tidyverse

The tidyverse is a collection of packages designed for data science that share a common philosophy.

Core packages include: `ggplot2`, `dplyr`, `tidyr`, `readr`, `tibble`, `purrr`, `stringr`, `forcats`, `lubridate`.

```
library(tidyverse)
```

R building blocks

Objects

An object is anything you store in R's memory with a name.

```r
x <- 10
name <- "Adriana"
numbers <- c(1, 2, 3, 4, 5)
```

Once created, you can use the name to access the value.

```r
numbers
```

```
[1] 1 2 3 4 5
```

Functions

A function is a reusable block of code that performs a task.

```
mean(numbers)
```

```
[1] 3
```

> Functions are called by their name followed by parentheses: `function_name()`.

Arguments

Arguments are the inputs you pass to a function.

```
round(3.14159, digits = 2)
```

```
[1] 3.14
```

Arguments can be named (`digits = 2`) or positional. Named arguments make your code clearer.

The assignment operator

Use `<-` to assign a value to an object.

```
age <- 30
age
```

```
[1] 30
```

You can also use `=`, but `<-` is the convention in R.

> **Keyboard shortcut**
>
> `Alt + -` (Windows/Linux) or `Option + -` (Mac) inserts `<-`.

The pipe operator

The pipe takes the result on the left and passes it as the first argument to the function on the right.

Without a pipe:

```
round(mean(c(1, 2, 3, 4, 5)), digits = 2)
```

```
[1] 3
```

With a pipe, the same operation reads left to right.

Two pipe flavors

magrittr pipe `%>%`

Comes from the `magrittr` package, loaded with the tidyverse.

```
c(1, 2, 3, 4, 5) %>%
  mean() %>%
  round(digits = 2)
```

```
[1] 3
```

base pipe `|>`

Built into R since version 4.1 (2021). No package needed.

```
c(1, 2, 3, 4, 5) |>
  mean() |>
  round(digits = 2)
```

```
[1] 3
```

Which pipe to use?

- For new code, prefer the base pipe `|>`: no dependencies, part of R itself
- You will see `%>%` in older code and tutorials: they behave almost the same for our purposes

> **Keyboard shortcut**
>
> `Ctrl + Shift + M` (Windows/Linux) or `Cmd + Shift + M` (Mac) inserts a pipe. You can set which one in RStudio preferences.

Introducing dplyr

What is dplyr?

dplyr is the tidyverse package for **data processing**.

It gives you a consistent set of verbs to manipulate data frames: filter rows, select columns, create new variables, compute summaries, and more.

The four dplyr rules

1. The **first argument** is always a data frame
2. The **next arguments** describe what to do with that data frame
3. It **returns a data frame**
4. It **does not modify** the original data frame

> These rules are what make dplyr play beautifully with the pipe.

Rule 1 and 2: data first, then instructions

```r
filter(starwars, species == "Droid")
```

```
# A tibble: 6 × 14
  name   height  mass hair_color skin_color  eye_color birth_year
sex   gender
  <chr>   <int> <dbl> <chr>      <chr>       <chr>           <dbl>
<chr> <chr>
1 C-3PO     167    75 <NA>       gold        yellow            112
none  masculi…
2 R2-D2      96    32 <NA>       white, blue red                33
none  masculi…
3 R5-D4      97    32 <NA>       white, red  red               NA
none  masculi…
4 IG-88     200   140 none       metal       red               15
```

With the pipe, the data frame moves out of the call:

```r
starwars |>
  filter(species == "Droid")
```

Rule 3: always returns a data frame

This means you can chain dplyr functions with the pipe.

```r
starwars |>
  filter(species == "Human") |>
  select(name, height, mass)
```

```
# A tibble: 35 × 3
   name               height  mass
   <chr>               <int> <dbl>
 1 Luke Skywalker        172    77
 2 Darth Vader           202   136
 3 Leia Organa           150    49
 4 Owen Lars             178   120
 5 Beru Whitesun Lars    165    75
 6 Biggs Darklighter     183    84
 7 Obi-Wan Kenobi        182    77
 8 Anakin Skywalker      188    84
 9 Wilhuff Tarkin        180    NA
```

Rule 4: does not modify the object

```r
starwars |>
  filter(species == "Droid")
```

```
# A tibble: 6 × 14
  name   height  mass hair_color skin_color  eye_color birth_year
sex   gender
  <chr>   <int> <dbl> <chr>      <chr>       <chr>          <dbl>
<chr> <chr>
1 C-3PO     167    75 <NA>       gold        yellow           112
none  masculi…
2 R2-D2      96    32 <NA>       white, blue red               33
none  masculi…
3 R5-D4      97    32 <NA>       white, red  red               NA
none  masculi…
4 IG-88     200   140 none       metal       red               15
```

The `starwars` object is unchanged. To keep the result, assign it to a new object.

```r
droids <- starwars |>
  filter(species == "Droid")
```

The dplyr verbs

`filter()`: keep rows that match a condition

```r
starwars |>
  filter(height > 200)
```

```
# A tibble: 10 × 14
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>          <dbl>
<chr> <chr>
 1 Darth V…    202   136 none       white      yellow          41.9
male  mascu…
 2 Chewbac…    228   112 brown      unknown    blue           200
male  mascu…
 3 Roos Ta…    224    82 none       grey       orange          NA
male  mascu…
 4 Rugor N…    206    NA none       green      orange          NA
```

`filter()` with comparison operators

Operator	Meaning
`==`	equal to
`!=`	not equal to
`<`, `<=`	less than (or equal)
`>`, `>=`	greater than (or equal)
`%in%`	value is in a set
`is.na()`	missing value

`filter()`: equality and inequality

```r
starwars |>
  filter(eye_color == "blue")
```

```
# A tibble: 19 × 14
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>          <dbl>
<chr> <chr>
 1 Luke Sk…    172  77   blond      fair       blue            19
male  mascu…
 2 Owen La…    178 120   brown, gr… light      blue            52
male  mascu…
 3 Beru Wh…    165  75   brown      light      blue            47
fema… femin…
 4 Anakin …    188  84   blond      fair       blue            41.9
```

`filter()` with `%in%`

Use `%in%` to match **several values** at once.

```
starwars |>
  filter(species %in% c("Droid", "Wookiee", "Ewok"))
```

```
# A tibble: 9 × 14
  name      height  mass hair_color skin_color eye_color birth_year
sex   gender
  <chr>      <int> <dbl> <chr>      <chr>      <chr>           <dbl>
<chr> <chr>
1 C-3PO        167    75 <NA>       gold       yellow            112
none  mascu…
2 R2-D2         96    32 <NA>       white, bl… red                33
none  mascu…
3 R5-D4         97    32 <NA>       white, red red               NA
none  mascu…
4 Chewbacca    228   112 brown      unknown    blue             200
```

`filter()` with multiple conditions

Combine conditions with `&` (and), `|` (or), or a comma (same as `&`).

```r
starwars |>
  filter(species == "Human", height > 180)
```

```
# A tibble: 15 × 14
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl>
<chr> <chr>
 1 Darth V…    202 136   none       white      yellow           41.9
male  mascu…
 2 Biggs D…    183  84   black      light      brown            24
male  mascu…
 3 Obi-Wan…    182  77   auburn, w… fair       blue-gray        57
male  mascu…
 4 Anakin …    188  84   blond      fair       blue             41.9
```

`select()`: keep or drop columns

```r
starwars |>
  select(name, height, mass)
```

```
# A tibble: 87 × 3
   name               height  mass
   <chr>               <int> <dbl>
 1 Luke Skywalker        172    77
 2 C-3PO                 167    75
 3 R2-D2                  96    32
 4 Darth Vader           202   136
 5 Leia Organa           150    49
 6 Owen Lars             178   120
 7 Beru Whitesun Lars    165    75
 8 R5-D4                  97    32
 9 Biggs Darklighter     183    84
```

`select()` with a minus sign

Use `-` to **drop** columns.

```
starwars |>
  select(-films, -vehicles, -starships)
```

```
# A tibble: 87 × 11
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl>
<chr> <chr>
 1 Luke Sk…    172    77 blond      fair       blue             19
male  mascu…
 2 C-3PO       167    75 <NA>       gold       yellow          112
none  mascu…
 3 R2-D2        96    32 <NA>       white, bl… red              33
none  mascu…
 4 Darth V…    202   136 none       white      yellow           41.9
```

`select()` with a range

Use `:` to select a **range** of consecutive columns.

```
starwars |>
  select(name:eye_color)
```

```
# A tibble: 87 × 6
   name               height  mass hair_color    skin_color
eye_color
   <chr>               <int> <dbl> <chr>         <chr>
 1 Luke Skywalker        172    77 blond         fair
 2 C-3PO                 167    75 <NA>          gold
 3 R2-D2                  96    32 <NA>          white, blue
 4 Darth Vader           202   136 none          white
 5 Leia Organa           150    49 brown         light
 6 Owen Lars             178   120 brown, grey   light
 7 Beru Whitesun Lars    165    75 brown         light
 8 R5-D4                  97    32 <NA>          white, red
```

`select()` with helper functions

Functions like `starts_with()`, `ends_with()`, and `contains()` match column names by pattern.

```
starwars |>
  select(name, starts_with("h"))
```

```
# A tibble: 87 × 4
   name               height hair_color    homeworld
   <chr>               <int> <chr>         <chr>
 1 Luke Skywalker        172 blond         Tatooine
 2 C-3PO                 167 <NA>          Tatooine
 3 R2-D2                  96 <NA>          Naboo
 4 Darth Vader           202 none          Tatooine
 5 Leia Organa           150 brown         Alderaan
 6 Owen Lars             178 brown, grey   Tatooine
 7 Beru Whitesun Lars    165 brown         Tatooine
 8 R5-D4                  97 <NA>          Tatooine
 9 Biggs Darklighter     183 black         Tatooine
```

Other useful helpers: `ends_with()`, `contains()`, `matches()`, `where()`.

arrange(): sort rows

```r
starwars |>
  arrange(height)
```

```
# A tibble: 87 × 14
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>          <dbl>
<chr> <chr>
 1 Yoda         66    17 white      green      brown            896
male  mascu…
 2 Ratts T…     79    15 none       grey, blue unknown           NA
male  mascu…
 3 Wicket …     88    20 brown      brown      brown              8
male  mascu…
 4 Dud Bolt     94    45 none       blue, grey yellow            NA
```

`arrange()` in descending order

Wrap a variable in `desc()` to sort from high to low.

```
1 starwars |>
2   arrange(desc(height))
```

```
# A tibble: 87 × 14
   name     height  mass hair_color skin_color eye_color birth_year
sex   gender
   <chr>     <int> <dbl> <chr>      <chr>      <chr>           <dbl>
<chr> <chr>
 1 Yarael …    264    NA none       white      yellow           NA
male  mascu…
 2 Tarfful     234   136 brown      brown      blue             NA
male  mascu…
 3 Lama Su     229    88 none       grey       black            NA
male  mascu…
 4 Chewbac…    228   112 brown      unknown    blue            200
```

`relocate()`: change column order

By default, `relocate()` moves columns to the **front**.

```
starwars |>
  relocate(species, homeworld)
```

```
# A tibble: 87 × 14
   species homeworld name           height  mass hair_color
skin_color eye_color
   <chr>   <chr>     <chr>           <int> <dbl> <chr>      <chr>
<chr>
 1 Human   Tatooine  Luke Skywalker    172    77 blond      fair
blue
 2 Droid   Tatooine  C-3PO             167    75 <NA>       gold
yellow
 3 Droid   Naboo     R2-D2              96    32 <NA>       white,
bl… red
 4 Human   Tatooine  Darth Vader       202   136 none       white
```

relocate() with .before and .after

```r
starwars |>
  relocate(species, .after = name)
```

```
# A tibble: 87 × 14
   name    species height  mass hair_color skin_color eye_color
birth_year sex
   <chr>   <chr>    <int> <dbl> <chr>      <chr>      <chr>
<dbl> <chr>
 1 Luke S… Human      172    77 blond      fair       blue
19   male
 2 C-3PO   Droid      167    75 <NA>       gold       yellow
112   none
 3 R2-D2   Droid       96    32 <NA>       white, bl… red
33   none
 4 Darth … Human      202   136 none       white      yellow
```

`rename()`: change column names

New name goes on the **left**, old name on the **right**.

```
starwars |>
  rename(character = name, planet = homeworld)
```

```
# A tibble: 87 × 14
   character       height  mass hair_color skin_color eye_color
birth_year sex
   <chr>            <int> <dbl> <chr>      <chr>      <chr>
<dbl> <chr>
 1 Luke Skywalker     172    77 blond      fair       blue
19   male
 2 C-3PO              167    75 <NA>       gold       yellow
112   none
 3 R2-D2               96    32 <NA>       white, bl… red
33   none
 4 Darth Vader        202   136 none       white      yellow
```

`mutate()`: create or modify columns

```r
starwars |>
  mutate(height_m = height / 100) |>
  select(name, height, height_m)
```

```
# A tibble: 87 × 3
   name               height height_m
   <chr>               <int>    <dbl>
 1 Luke Skywalker        172     1.72
 2 C-3PO                 167     1.67
 3 R2-D2                  96     0.96
 4 Darth Vader           202     2.02
 5 Leia Organa           150     1.5
 6 Owen Lars             178     1.78
 7 Beru Whitesun Lars    165     1.65
 8 R5-D4                  97     0.97
 9 Biggs Darklighter     183     1.83
```

mutate(): multiple columns at once

```r
starwars |>
  mutate(
    height_m = height / 100,
    bmi = mass / (height_m ^ 2)
  ) |>
  select(name, height_m, mass, bmi)
```

```
# A tibble: 87 × 4
   name                height_m  mass   bmi
   <chr>                  <dbl> <dbl> <dbl>
 1 Luke Skywalker          1.72    77  26.0
 2 C-3PO                   1.67    75  26.9
 3 R2-D2                   0.96    32  34.7
 4 Darth Vader             2.02   136  33.3
 5 Leia Organa             1.5     49  21.8
 6 Owen Lars               1.78   120  37.9
 7 Beru Whitesun Lars      1.65    75  27.5
 8 R5-D4                   0.97    32  34.0
 9 Biggs Darklighter       1.83    84  25.1
```

`transmute()`: mutate and keep only new columns

```r
starwars |>
  transmute(
    name,
    height_m = height / 100,
    bmi = mass / (height / 100) ^ 2
  )
```

```
# A tibble: 87 × 3
   name                height_m   bmi
   <chr>                  <dbl> <dbl>
 1 Luke Skywalker          1.72  26.0
 2 C-3PO                   1.67  26.9
 3 R2-D2                   0.96  34.7
 4 Darth Vader             2.02  33.3
 5 Leia Organa             1.5   21.8
 6 Owen Lars               1.78  37.9
 7 Beru Whitesun Lars      1.65  27.5
 8 R5-D4                   0.97  34.0
```

```
9 Biggs Darklighter        1  83   25  1
```

`transmute()` is like `mutate()` followed by `select()`, all in one step.

summarize(): collapse to summary statistics

```r
starwars |>
  summarize(
    avg_height = mean(height, na.rm = TRUE),
    avg_mass = mean(mass, na.rm = TRUE),
    n = n()
  )
```

```
# A tibble: 1 × 3
  avg_height avg_mass     n
       <dbl>    <dbl> <int>
1       175.     97.3    87
```

> summarize() returns one row per group, with the columns you define.

`group_by()`: compute by group

`group_by()` on its own does not change the data, but it tells dplyr that subsequent operations should run **per group**.

```r
starwars |>
  group_by(species) |>
  summarize(
    n = n(),
    avg_height = mean(height, na.rm = TRUE)
  )
```

```
# A tibble: 38 × 3
   species       n avg_height
   <chr>     <int>      <dbl>
 1 Aleena        1         79
 2 Besalisk      1        198
 3 Cerean        1        198
 4 Chagrian      1        196
 5 Clawdite      1        168
 6 Droid         6        131.
 7 Dug           1        112
```

```
8 Ewok          1        88
9 Geonosian     1       183
```

group_by() with multiple variables

```r
starwars |>
  group_by(species, gender) |>
  summarize(n = n(), .groups = "drop")
```

```
# A tibble: 42 × 3
   species   gender        n
   <chr>     <chr>     <int>
 1 Aleena    masculine     1
 2 Besalisk  masculine     1
 3 Cerean    masculine     1
 4 Chagrian  masculine     1
 5 Clawdite  feminine      1
 6 Droid     feminine      1
 7 Droid     masculine     5
 8 Dug       masculine     1
 9 Ewok      masculine     1
```

`count()`: shortcut for grouped counts

`count(x)` is equivalent to `group_by(x) |> summarize(n = n())`.

```
starwars |>
  count(species)
```

```
# A tibble: 38 × 2
   species       n
   <chr>     <int>
 1 Aleena        1
 2 Besalisk      1
 3 Cerean        1
 4 Chagrian      1
 5 Clawdite      1
 6 Droid         6
 7 Dug           1
 8 Ewok          1
 9 Geonosian     1
```

count() with sorting

```r
starwars |>
  count(species, sort = TRUE)
```

```
# A tibble: 38 × 2
   species      n
   <chr>    <int>
 1 Human       35
 2 Droid        6
 3 <NA>         4
 4 Gungan       3
 5 Kaminoan     2
 6 Mirialan     2
 7 Twi'lek      2
 8 Wookiee      2
 9 Zabrak       2
```

The big picture

Verbs at a glance

Verb	What it does
`filter()`	Keep rows that match a condition
`select()`	Keep or drop columns
`arrange()`	Sort rows
`relocate()`	Reorder columns
`rename()`	Rename columns
`mutate()`	Create or modify columns
`transmute()`	Create columns, drop the rest
`summarize()`	Collapse to summary statistics
`group_by()`	Compute per group
`count()`	Shortcut for grouped counts

Putting it all together

```r
starwars |>
  filter(!is.na(height), !is.na(mass)) |>
  mutate(bmi = mass / (height / 100) ^ 2) |>
  group_by(species) |>
  summarize(avg_bmi = mean(bmi), n = n()) |>
  arrange(desc(avg_bmi))
```

```
# A tibble: 32 × 3
   species        avg_bmi     n
   <chr>            <dbl> <int>
 1 Hutt             443.      1
 2 Vulptereen        50.9     1
 3 Yoda's species    39.0     1
 4 Kaleesh           34.1     1
 5 Droid             32.7     4
 6 Dug               31.9     1
 7 Trandoshan        31.3     1
 8 Sullustan         26.6     1
 9 Zabrak            26.1     1
```

Questions?